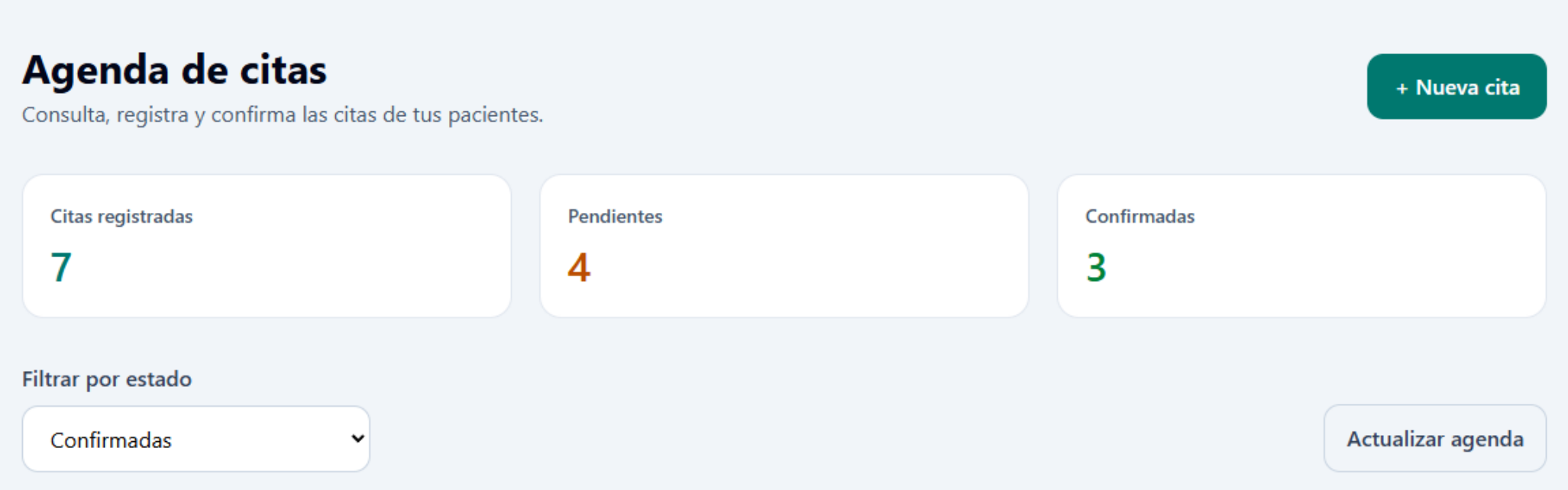
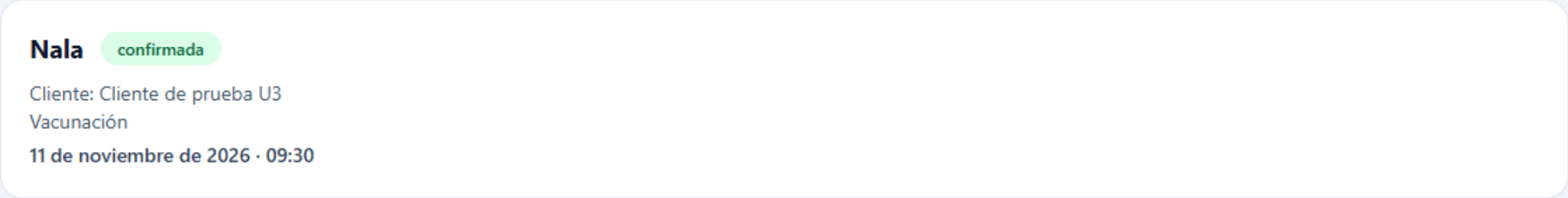
Actividad Sumativa 1 - Aplicación y MySQL

Alan Daniel de la Cruz Loeza | Aplicaciones Web | Unidad 3 | 10 de noviembre de 2026

React consume el API de Express y guarda las citas en **MySQL, base veterinaria_db, tabla citas**. Se utilizó la estructura compartida por el profesor en Teams. Los registros de clase y de prueba son ficticios.



Registro y confirmación: el formulario creó Nala mediante **POST /api/citas** (201, id 7). Después, **PATCH /api/citas/7/estado** devolvió 200 y la cita quedó confirmada.

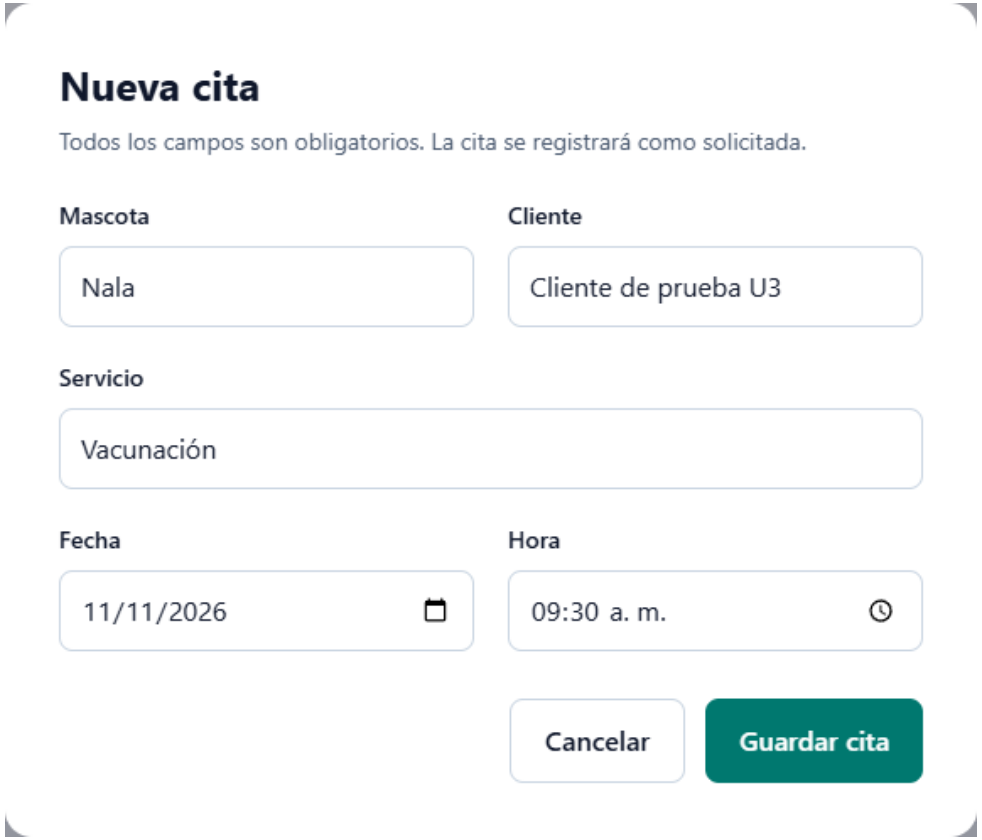


Registro y validaciones visibles

Alan Daniel de la Cruz Loeza | Aplicaciones Web | Unidad 3 | 10 de noviembre de 2026

El formulario solicita mascota, cliente, servicio, fecha y hora. La longitud de mascota coincide con **VARCHAR(80)** del SQL de clase. El backend valida los datos antes de ejecutar el INSERT.

Formulario con los datos de Nala



Validación de campos obligatorios



También se comprobó el horario ocupado: MySQL devuelve un conflicto y el API responde **409**. El mensaje es visible y el formulario conserva lo escrito. No hubo errores de JavaScript ni desbordamiento horizontal en la revisión móvil de 390 px.

Persistencia y pruebas en MySQL

Alan Daniel de la Cruz Loeza | Aplicaciones Web | Unidad 3 | 10 de noviembre de 2026

Prueba realizada: se consultaron las siete citas por el API, se cerró ese proceso de Express y se inició uno nuevo. La respuesta posterior coincidió con la anterior y con un **SELECT directo de MySQL**, incluidos IDs y estados. La tabla InnoDB devolvió **OK** en CHECK TABLE.

ID	Mascota	Fecha	Hora	Estado después del reinicio
1	Max	2026-10-28	09:00	solicitada
2	Voyka	2026-10-28	13:00	solicitada
3	Firulais	2026-10-28	15:00	solicitada
4	Rocky	2026-10-28	12:00	solicitada
5	Luna	2026-11-10	10:30	confirmada
6	Max	2026-11-10	11:30	confirmada
7	Nala	2026-11-11	09:30	confirmada

Verificación técnica: lint y compilación de frontend/backend correctos; **19/19 pruebas pasadas**. Incluyen importación del SQL de Teams, preparación repetida sin duplicar datos, GET, POST, PATCH, PUT, DELETE, errores de entrada y dos altas simultáneas en el mismo horario.

Conexión comprobada: MySQL 8.4.13 | base veterinaria_db | tabla citas | GET /api/salud: baseDeDatos = MySQL, estado = conectada.
Archivos SQL: backend/database/profesor.sql y backend/database/respaldo_demostracion.sql.

Flujo para explicar: formulario -> fetch -> Express -> ruta -> controlador -> repositorio -> MySQL -> respuesta -> React.
Errores: asyncHandler usa .catch(next); el middleware central envía el código y el mensaje JSON.

Evidencias de ejecución y consultas: docs/evidencias. Guía de estudio: GUIA_PARA_EXPLICAR.md. La revisión individual del profesor requiere explicar el código con tus propias palabras.